Project - Low Level Design

on

Dockerized Sports AI Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AANCHAL MARATHE	EN22CS301015
02	AAHMAD BURHANI	EN22CS301010
03	ABHISHEK SANKHALA	EN22CS301035
04	YASHASVI MODI	EN22CS3011114
05	AKSHAT YADAV	EN22CS301095

Group Name: 10D1

Project Number: GAI-10

Industry Mentor Name:

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-2026

Table of Contents

1. Introduction

1.1. Scope of the document

1.2. Intended Audience

1.3. System overview

2. System Design

2.1. Application Design

2.2. Process Flow

2.3. Information Flow

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue

3. References

1. Introduction

This document provides a detailed Low-Level Design (LLD) of the **Dockerized Sports AI Service**, an intelligent sports planning assistant built using Python-based AI frameworks and deployed using modern DevOps practices.

The system leverages **agentic AI architecture** to process user queries related to sports data and planning. It integrates external APIs for real-time sports information and uses containerization technologies to ensure portability, scalability, and consistency across environments.

The application is deployed using Docker and orchestrated using Kubernetes on cloud infrastructure, ensuring high availability and efficient resource utilization.

1.1 Scope of the Document

The scope of this document includes the detailed design and implementation aspects of the system, covering:

- Internal architecture of the application components
- Interaction between frontend, backend, and AI agents
- API integrations and data flow
- Docker containerization strategy
- Kubernetes deployment configuration
- Environment configuration and dependency management

This document focuses on the **technical implementation details** and does not include high-level business requirements or project planning aspects.

1.2 Intended Audience

This document is intended for:

- **Developers** – to understand the system architecture and code-level design
- **DevOps Engineers** – to manage containerization, deployment, and scaling
- **Project Evaluators / Faculty** – to assess technical implementation and design quality
- **System Maintainers** – for troubleshooting and future enhancements

Basic knowledge of the following is recommended:

- Python programming
- Docker and containerization
- Kubernetes fundamentals
- REST APIs and web applications
- AI frameworks such as CrewAI and LangChain

1.3 System Overview

The system is a **Sports AI Assistant Application** that allows users to interact through a web interface and receive intelligent responses based on sports data.

Key Features:

- AI-powered query handling using agent-based reasoning
- Integration with external sports APIs for real-time data
- Interactive frontend built using Streamlit
- Secure handling of API keys and environment variables
- Containerized deployment using Docker
- Scalable orchestration using Kubernetes

High-Level Workflow:

1. User submits a query through the web interface
2. The request is processed by AI agents
3. Agents fetch data from external APIs if required
4. The system generates a contextual response using LLM
5. Response is displayed back to the user

Deployment Overview:

- Application is packaged as a Docker image
- Hosted on cloud infrastructure via Amazon Web Services
- Managed and scaled using Kubernetes

2. System Design

The system follows a modular and scalable architecture combining AI-driven processing with modern DevOps deployment practices. It is designed to ensure flexibility, maintainability, and efficient interaction between components.

3.1 Application Design

The application is built using a **three-layer architecture**:

1. Presentation Layer

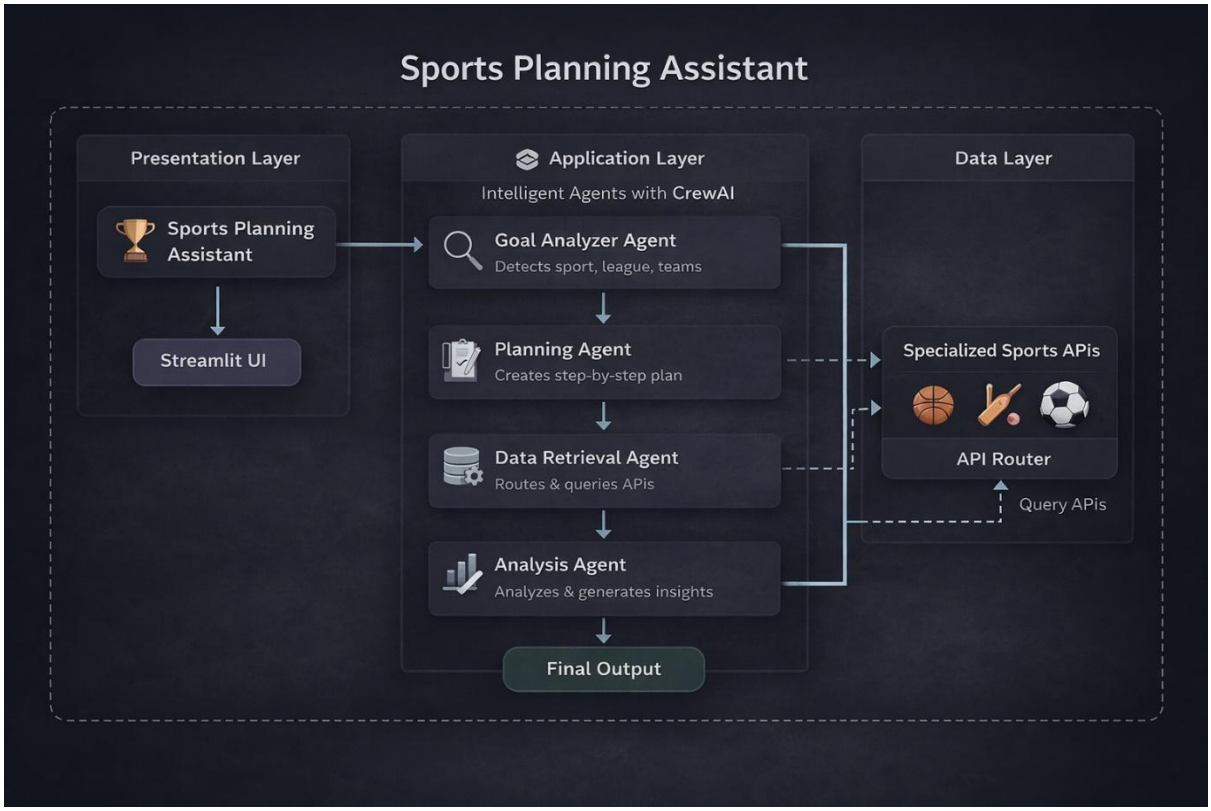
- Built using Streamlit
- Handles user input and displays AI-generated responses
- Provides an interactive UI for sports-related queries

2. Application (Logic) Layer

- Implements AI-based processing using:
  - CrewAI
  - LangChain
- Handles:
  - Query interpretation
  - Agent decision-making
  - Tool/API invocation

3. Data Layer

- Integrates external APIs like TheSportsDB API
- Retrieves real-time sports data
- Manages environment variables and API keys securely

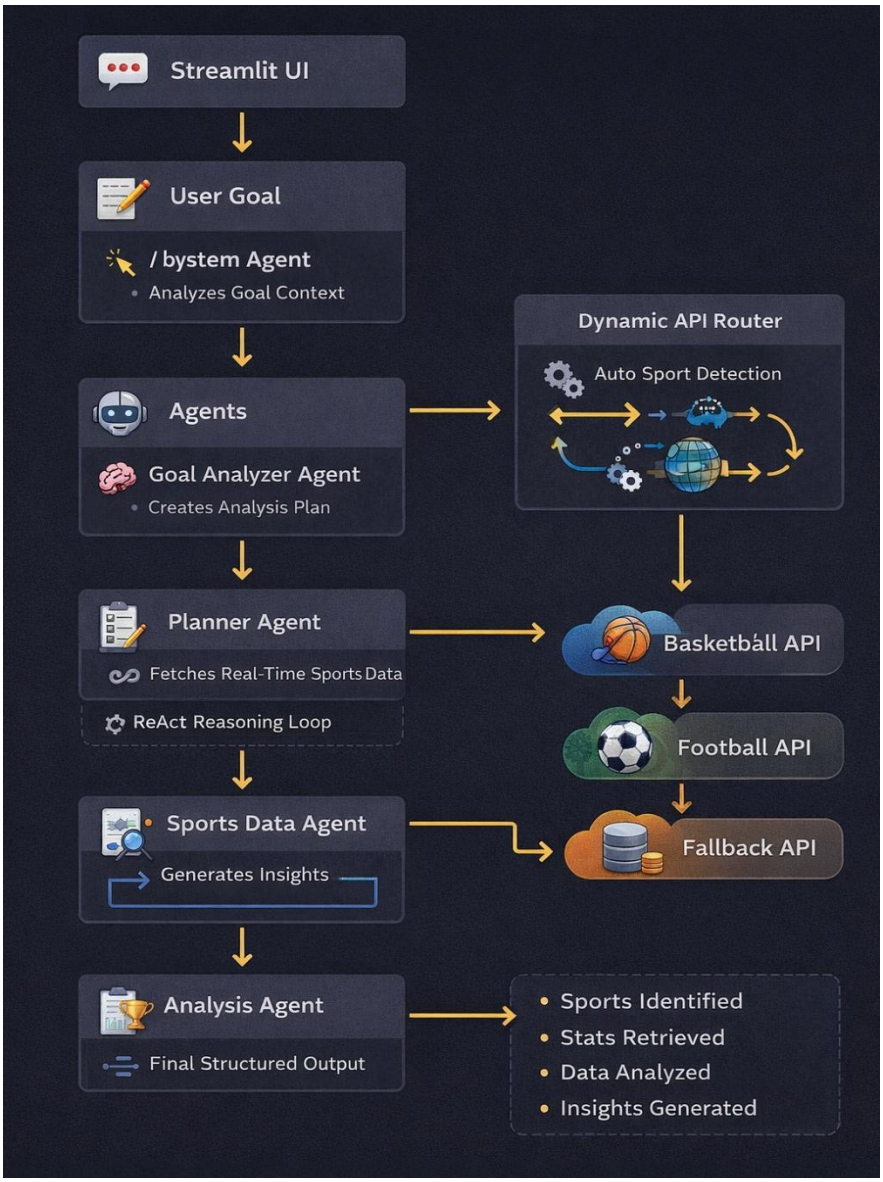


3.2 Process Flow

The process flow describes how a user request is handled:

1. User enters a query in the UI
2. Request is sent to the backend logic
3. AI agent interprets the query
4. If needed, external API is called
5. Data is processed using LLM
6. Response is generated
7. Output is displayed on UI

This flow ensures dynamic and intelligent responses using agent-based reasoning.



3.3 Information Flow

The information flow defines how data moves across the system:
User → Streamlit UI → AI Agent → External API → LLM → Response → UI
Flow Description:

- Input: User query
- Processing: AI agents + LLM
- External Data: Sports API response
- Output: Contextual response

This ensures seamless integration between user interaction and intelligent processing.

3.4 Components Design

The system is divided into the following core components:
1. Frontend Component

- Built using Streamlit

- Handles user interaction

2. AI Processing Component
   - CrewAI agents handle reasoning and decision-making
   - LangChain manages LLM workflows
3. API Integration Component
   - Communicates with TheSportsDB API
   - Fetches real-time sports data
4. Configuration Component
   - Uses .env file for managing secrets
   - Ensures secure API key handling
5. Containerization Component
   - Docker used for packaging the application
   - Ensures consistency across environments
6. Orchestration Component
   - Kubernetes manages deployment, scaling, and availability

3.5 Key Design Considerations

The following principles were considered while designing the system:

Scalability
- Kubernetes enables horizontal scaling

Portability
- Docker ensures application runs consistently across systems

Security
- API keys stored using environment variables
- Sensitive data not hardcoded

Modularity
- Separation of frontend, logic, and data layers

Performance
- Efficient API calls and lightweight containers

Maintainability
- Clear code structure and modular components

3.6 API Catalogue

The system uses the following APIs:

1. Sports Data API
- Name: TheSportsDB API
- Purpose: Fetch sports-related data (teams, matches, players)
- Method: GET
- Example Endpoint:

https://www.thesportsdb.com/api/v1/json/{API_KEY}/searchteams.php?t=Arsenal

2. LLM API
- Name: OpenAI API
- Purpose: Generate intelligent responses
- Method: POST
- Usage:

- Query understanding
- Response generation

3. Internal Agent Tools

- Used within CrewAI framework
- Purpose:
  - Decision making
  - Tool calling
  - Task execution

4. References

- CrewAI Documentation
- LangChain Documentation
- Streamlit Documentation
- TheSportsDB API
- OpenAI API
- Docker
- Kubernetes
- Amazon Web Services
- Python Official Documentation
- Git Documentation